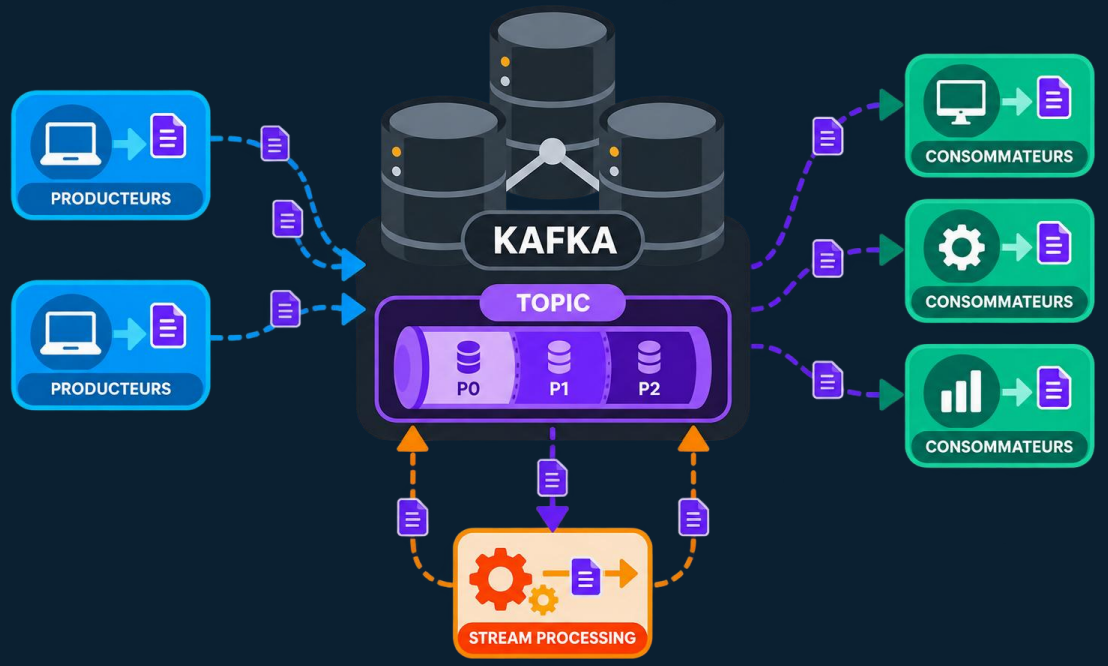
Stream Processing and EDA Resilience

Kafka Engineering Guide

From the first published message
to EDA resilience



A learning Sandbox …
Spring Boot · Nuxt · Kafka · Flink · Hazelcast

EDA architecture best practices

This guide presents a concrete project (with a Sandbox-style implementation) that integrates best practices.

01 Underestimation

Delivery teams generally underestimate the complexity of integrating with EDA.

02 Distributed Kafka

The distributed nature of the Kafka architecture must be taken into account.

03 External partners

Our partners have very little expertise in this area — this is simply a fact.

04 Sandbox-style project

A concrete project that demonstrates an EDA architecture implementation.

05 Introduction and concept walkthrough with implementation

We will present best practices for implementing a resilient architecture with concrete examples — through the learning Sandbox.

More than 40 journey slides + a few orientation and deep-dive appendices.

Six steps for knowledge transfer

One common thread: publish a request, produce a business effect, then control/corroborate that effect.

01 Discover the Sandbox

Application, data, and the journey of a request (message).

02 Understand EDA/Kafka

Topics, partitions, keys, and business ordering.

03 Make effects reliable

Delivery, transactions, idempotence, DLT, and Outbox.

04 Explore Flink

Checkpoint, savepoint, and runtime boundaries.

05 Operate and distribute

Parallelism, shutdown, probes, and elasticity.

06 Prove and choose

Observability, governance, corroboration, and TCO/ROI.

GITHUB Project: https://github.com/rdemers/org-rd-fullstack-springboot-eda

Publishing a message is not enough to build an EDA platform

EDA: event-driven architecture. Transport makes exchange possible; by itself, it does not guarantee the reliability of the effects.

01 **Transport**

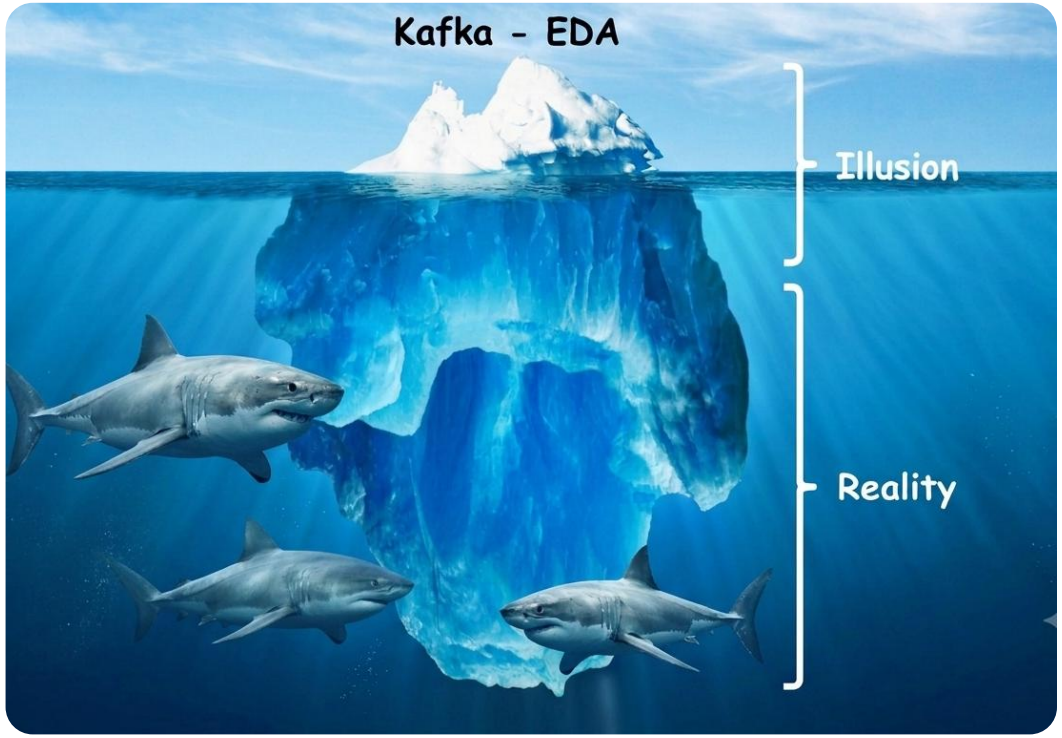
Publish, store, and consume normalized messages.

02 **Accuracy**

Order, deduplicate, and validate effects (idempotence).

03 **Trust**

Observe, govern, control, and corroborate the data.



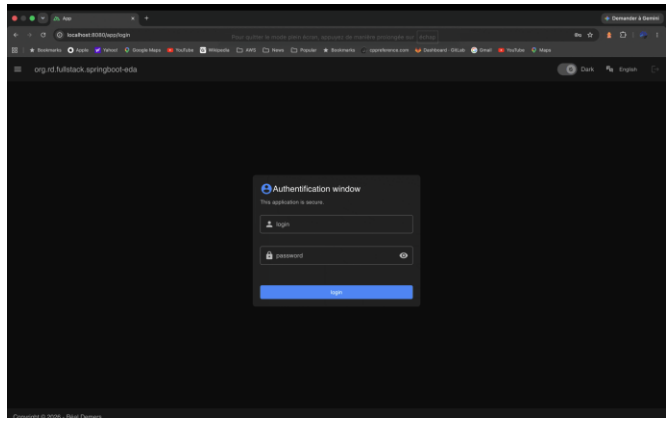
Our sharks: Bronze, silver, and gold — our medallion architecture specialists.

A standalone application for observing effects/trade-offs/decisions

The static Nuxt frontend and the Spring Boot backend are packaged in a single deployable artifact.

Browser

Nuxt interface



STANDALONE APPLICATION ARTIFACT

Spring Boot + Nuxt

BFF API and static resources

Configuration, transactions and business logic

Kafka

Flink

Hazelcast

HSQLDB

Educational Sandbox: an execution convenience, not a production topology.

A request connects a person, a product, and an inventory

Processing must evolve the balance, inventory, and request result coherently.

01 Key, Latency, Lock

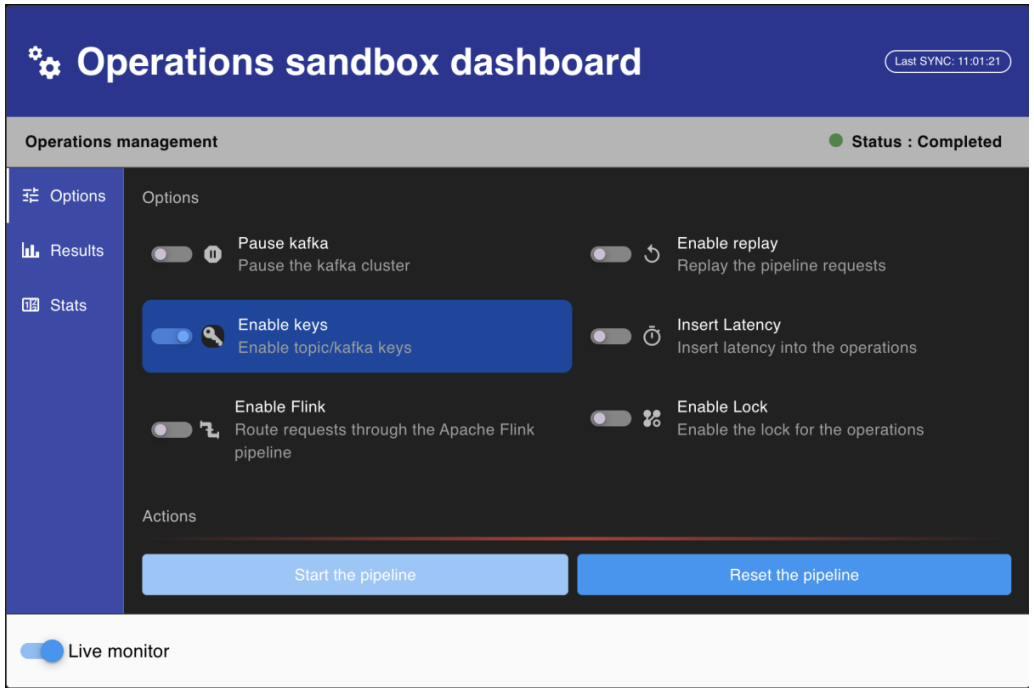
Make the effect of parallelism visible.

02 Pause, Replay

Observe progress and recovery.

03 Flink option

Flink alternative for processing.



PERSON

The buyer and their balance

PRODUCT

The product and its unit price

REQUEST

Operations to perform

INVENTORY

Available inventory

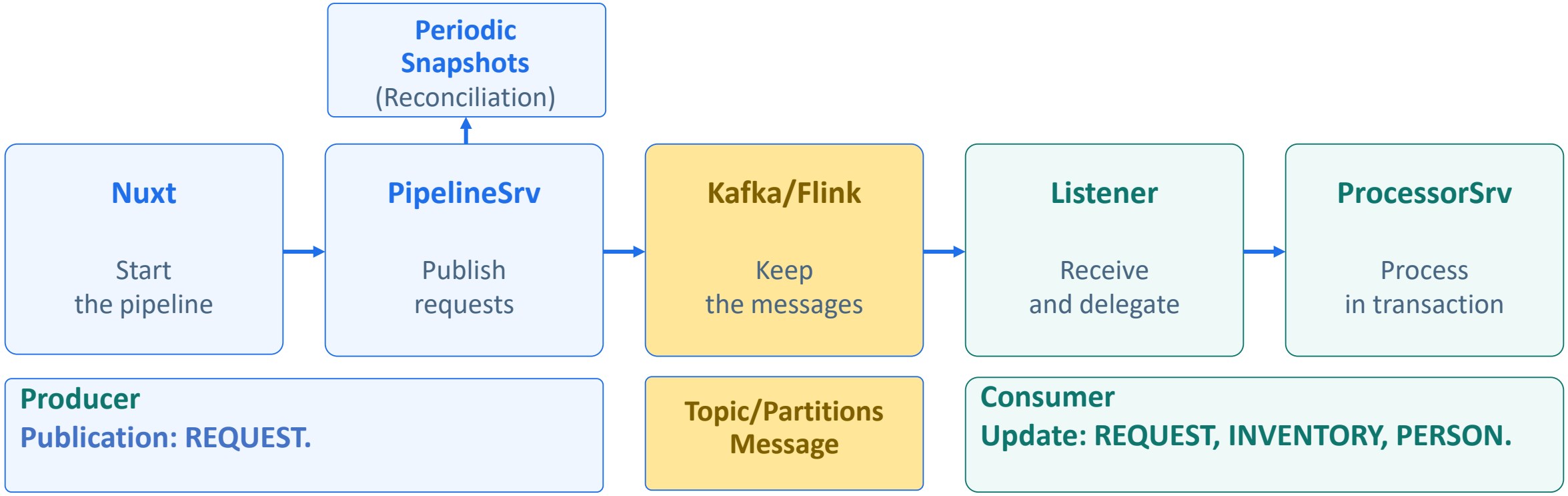
JRN_EVENT

Event consumption (idempotence)

Journey invariant: inventory must never become negative. The operations screen is the experimentation station.

Follow a request (message) end to end

Each component carries a distinct responsibility; the validation boundaries come next.



A received request is not immediately a validated business effect.

Data normalization

The topic carries messages from a producer. A message has a payload and metadata. Normalization must not depend on the technology and must be understandable by all present and future consumers.

01 Normalization

Normalization aims to make the message accessible/understandable to consumers — durable over time.

02 Metadata

Allows the consumer to know the context and obtain the unique identifier. Alternative: payload hash … if there is a timestamp field.

03 Integrity

A consumer must be able to validate message integrity BEFORE processing.



Technology: metadata must not depend on MOM-specific features.

Neither a universal topic nor uncontrolled multiplication

The goal is to find a balance around aggregates, responsibilities, and contracts.

Fat pipe



A topic for heterogeneous events.

Risk: unclear contract and unnatural (or impossible) key.

Makes message consumption exponentially more complex.

Thin pipes



Specialized topics by type.

Trade-off: inter-topic ordering and operations.

Ideal for selecting a partition key.

By aggregate

An explicit intent and contract.

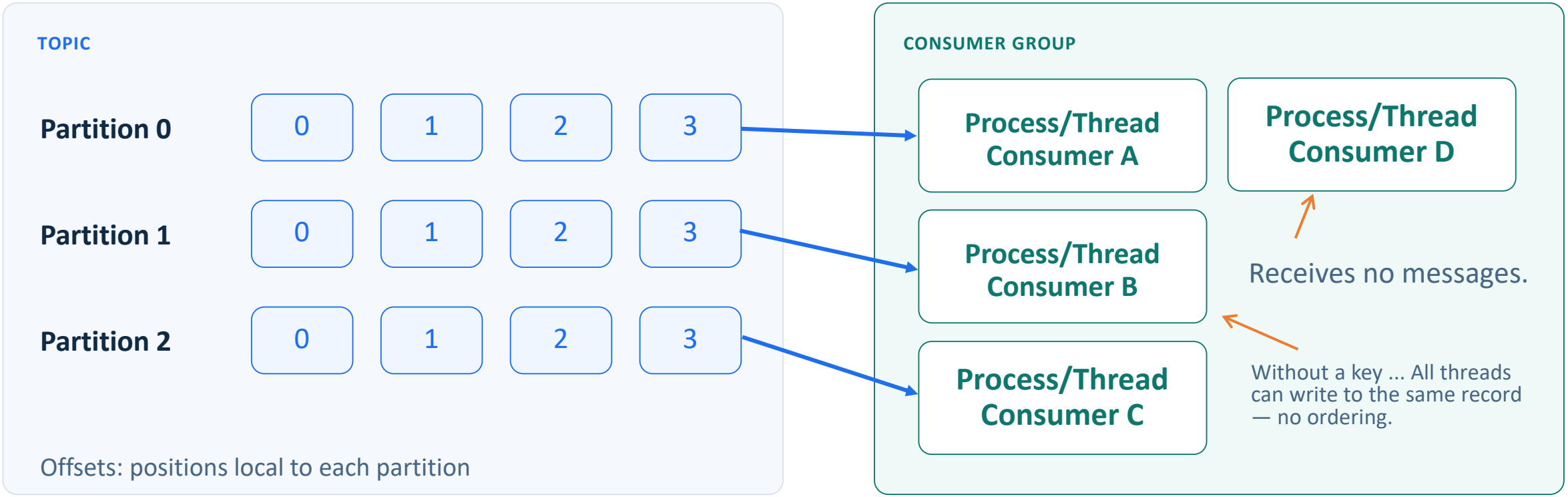
Group events whose order is related.

May be difficult to determine a good partition key.

A topic is an intentional contract, not just a transport pipe.

A Kafka topic … Partitions allocate horizontal elasticity

The topic organizes messages; the partition provides order (when keyed); the offset marks a position; the group shares the work.

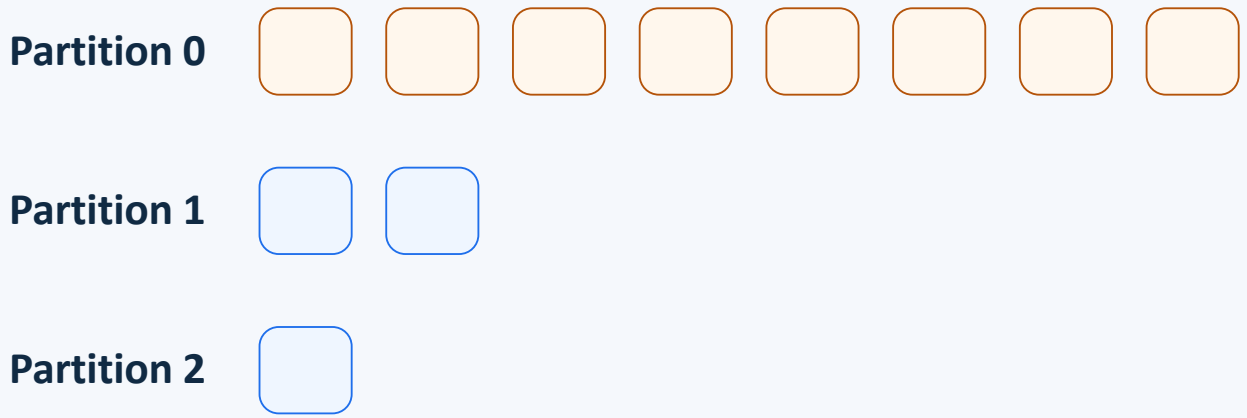


Order is guaranteed only within a partition (when a partition key is used), not globally across all partitions.

An imbalanced key can neutralize horizontal elasticity

Throughput can remain limited by a single partition, even when other consumers are available.

IMBALANCE DIAGRAM



A dominant key concentrates the work.

01 **Measure distribution**

Observe lag by partition, not just the total.

02 **Locate the bottleneck**

Key, lock, database, or external resource.

03 **EKS – Kafka rebalance**

An inactive (underactive) pod is removed, continuously causing partition rebalances toward other pods ... Ouch!

Distributing more is useful only if business ordering and invariants remain preserved.

The Kafka key must express the desired business ordering

In the project, the “productId” key provides message ordering for the inventory invariant.

Without a key

The same product may appear in several partitions.

Multiple threads may act on the same inventory.

With “productId” as the key

Product requests go to the same partition.

The direct path preserves sequential processing by product.

However, this ordering does not address synchronization of the person balance.

Design decision: order events where the business requires it.

Only one option changes; the invariant can break

The two reports in the project documentation use the same initial inventory and the same requests, with or without a key.

WITH KEY

Banana : 0

ID	Code	Description	Qty/Qté
0	Apple	Mcinstosh Apple	0
1	Banana	Banana split	0
2	Steak	T-Bone Steak 4pack	10
3	Oats	Quaker Quick Oats	80

Illustrated result: non-negative inventory.

WITHOUT KEY or key on “personId”.

Banana : -10

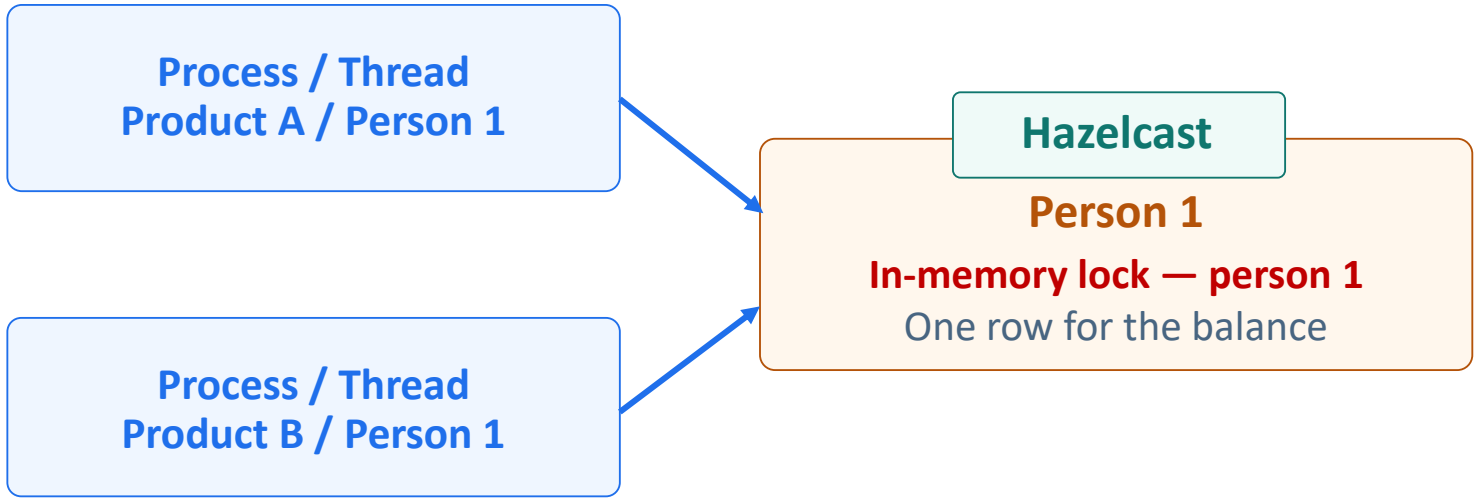
ID	Code	Description	Qty/Qté
0	Apple	Mcinstosh Apple	120
1	Banana	Banana split	-10
2	Steak	T-Bone Steak 4pack	160
3	Oats	Quaker Quick Oats	170

Illustrated result: silent overselling.

Documentation excerpts: the comparison must focus on business effects, not only on data flow.

Protect the person and the product

The Hazelcast lock is indexed on PERSON_ID; the INVENTORY row is ordered by PRODUCT_ID.



The race

Read the person balance
Validate the balance and modify it.

Two processing flows can pass the same balance validation.

Adding “personId” to the current key will not work!

The scope of a protection must match the invariant it is intended to protect.

Alternative - Protect PERSON or INVENTORY using SQL locking

The SQL approach ... an alternative that does not preserve ordering.

Pessimistic locking

Lock the row on read so others wait. In SQL: SELECT ... FOR UPDATE.

Process waiting on the database. “sleep mode”.

Less elasticity and an impact on the lifecycle of a pod.

Optimistic locking



A @Version column; the update succeeds only if the version is unchanged, otherwise the application must retry.

Spins in circles ... consumes $$$ resources. Fight between pods (Process/Thread)!!!

Conditional update

Delegate the balance check to the database write (where clause).

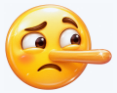
Business logic moves down into the SQL query — breaks the JPA cache.

Protects only the invariants explicitly guarded, without automatically protecting the entire entity from all concurrent changes.

Breaks horizontal elasticity, generates DLT, consumes resources, and can cause repeated retries.
This alternative does not preserve ordering!!!

Three guarantees, but always a boundary to define

The question is not only how many times a message arrives, but which effects become visible.

Guarantee	Principle	Trade-off
At-most-once	Validate the position before processing.	Loss is possible and will occur!!!
At-least-once 	Validate after successful work.	Duplicates are possible. Idempotence must be implemented.
Exactly-once 	Atomic Kafka results and source offsets.	**External effects are not included. Note: checkpoint!**

```java
public class KafkaPipelineListener {

    @KafkaListener(id = KafkaConstants.CST_LISTENER_PROCESSOR,
                   topics = KafkaConstants.CST_TOPIC_KAFKA_REQ,
                   groupId = KafkaConstants.CST_TOPIC_GROUP,
                   containerFactory = "kafkaListenerContainerFactory")
    public void listen(ConsumerRecord<String, String> record, Acknowledgment
                @Header(KafkaHeaders.RECEIVED_TOPIC)     String topic,
                @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                @Header(KafkaHeaders.OFFSET)             long offset) throws

        // Per-record processing (per-client lock + transactional processor
        // is shared with the Flink sink path; see PipelineSrv.handle(). Any
        // the container's DefaultErrorHandler, which retries with a bounded
        // routes the record to the DLT. The JPA transaction in the process
        // so no partial DB state is committed.
        pipeline.handle(record.value());

        // Commit the offset only after a successful, committed processing.
        // (at-least-once: a crash before this point replays the message.)
        ack.acknowledge();
    }
```

At-least-once.

Sandbox project choice: at-least-once + idempotent business processing.

Three different mechanisms work together

Producer idempotence, Kafka transaction, and consumer isolation do not answer the same question.

Idempotent producer

Avoid duplicates caused by producer retries.

enable.idempotence=true

Kafka transaction

Commit or abort a set of publications in Kafka.

commit / abort

Isolated consumer

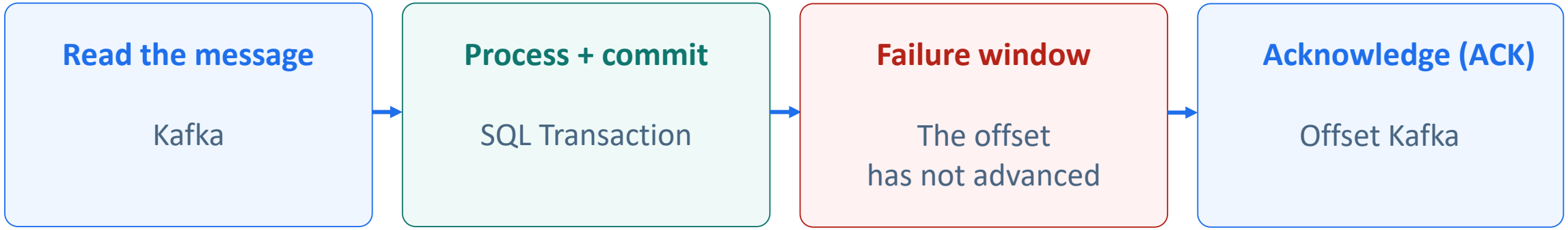
Transactional messages: read only after commit.

isolation.level= read_committed

These Kafka mechanisms do not replace SQL effect atomicity or idempotence.

The business commit must precede the acknowledgement

A failure between the two commits causes redelivery: that is exactly why idempotence is needed.



Offset before SQL

Risk: the business effect may be lost.

- Failure/Shutdown – No redelivery.
- Exception – No redelivery.

SQL before offset

Risk: the effect may be attempted again.

- Failure/Shutdown – Redelivery.
- Exception – Redelivery.

A delivery duplicate must become a non-event for the data — processing MUST BE IDEMPOTENT.

Recognize the event (the message) — Idempotence

The same event delivered multiple times must apply the business effect only once.

ONE SQL TRANSACTION

1 **Verify / record the identity**

JRN_EVENT : (CONSUMER_ID, EVENT_ID)

2 **Apply, then record the outcome**

INVENTORY + balance + REQUEST + JRN_EVENT

A uniqueness constraint remains the authority for deduplication under concurrency. It is not the commit. Keep identity, processing type, and result distinct.

```java
public class ProcessorSrv {
    public ProcessorSrv(JrnEventRepository jrnEventRepository, InventoryRepository inventoryRepository,

    @Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.READ_COMMITTED)
    public void process(EventMessage<EventHeader, Request, EventFooter> message,
        boolean latence) throws Exception {

        SanityResult sanity = sanityCheck(message);
        if (! sanity.proceed())
            return; // Already processed, or exception thrown (rollback and retry).

        final Request  request  = message.getPayload();
        final JrnEvent jrnEvent = sanity.jrnEvent();

        // Get inventory for the product.
        Optional<Inventory> inventoryOptional = inventoryRepository.findByProductId(request.getProductId());
        if (inventoryOptional.isEmpty()) { // Big problem here.
            finalizeOutcome(request, jrnEvent, Result.ERROR, false);
            return;
        }
```

A single transaction.

Verify idempotence.

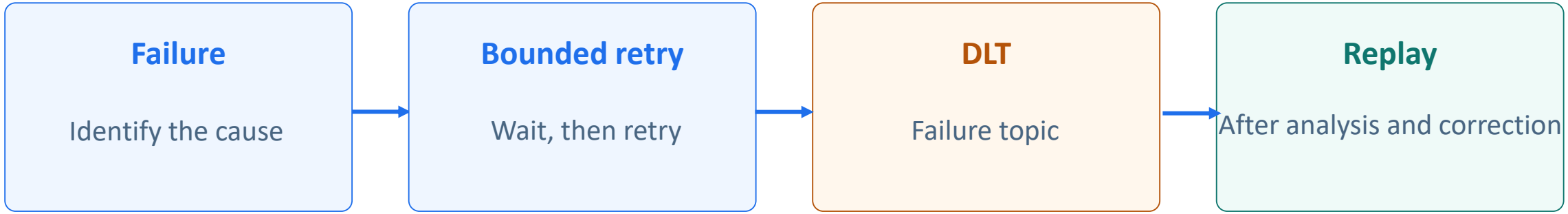
Journal reference for the update.

Thus ... start of the business logic.

Deduplication marker and business effects: commit together, or roll back together.

An error must follow an explicit path

Retry a transient incident, isolate a persistent failure, then organize a controlled replay.



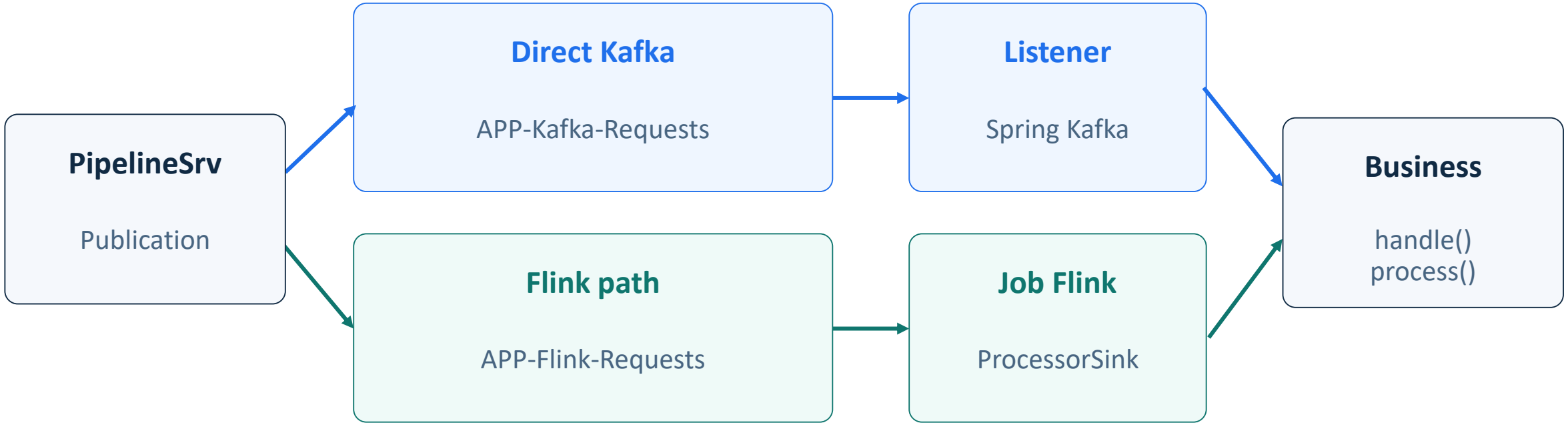
Context
Event identity, topic, partition, offset, and failure cause.

Procedure
How to perform the replay … A message in DLT is not a final state.

A DLT must have an owner and a handling procedure (do not turn the DLT into a black hole).

Flink changes the transport, not the business intent

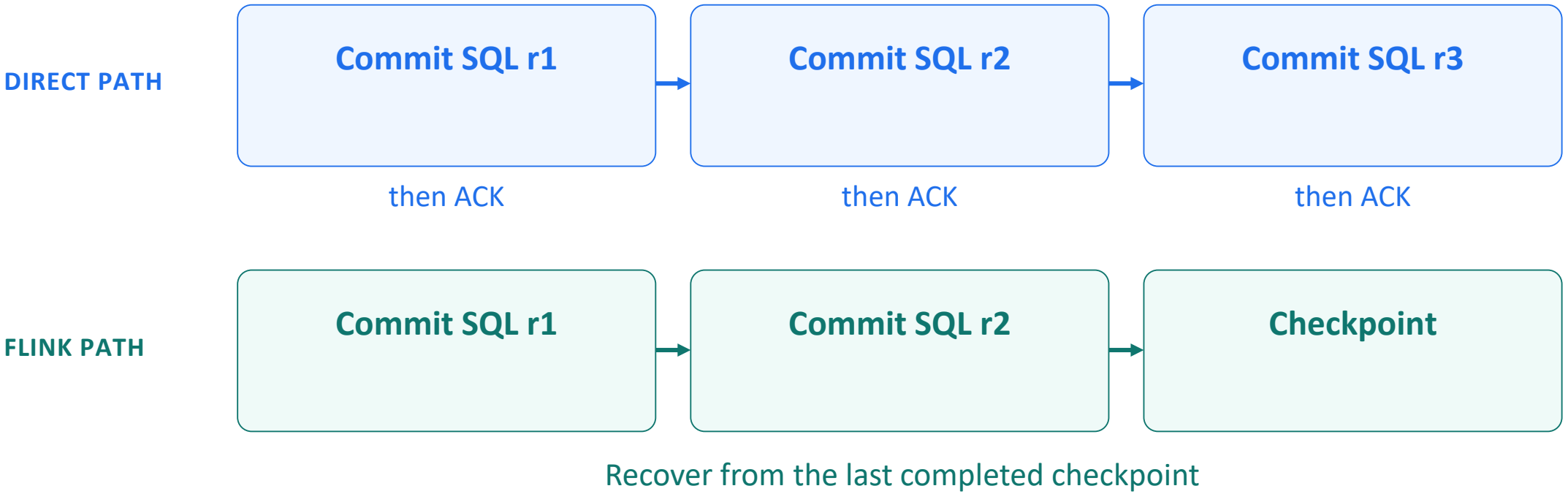
The Flink button selects the publication topic; both paths converge to the same processing.



The same SQL effect, but different progress and recovery boundaries.

A Flink checkpoint is not a SQL commit

The last completed checkpoint sets the Flink recovery point, not the state already committed in SQL.



Note: idempotence must also be considered for counters and side effects (see Dashboard).

The in-JVM bridge is not a distributed integration

A remote TaskManager shares neither Spring beans, nor JPA repositories, nor the application’s static state.

Local Sandbox

MiniCluster and Spring share one JVM.

The bridge resolves the service at runtime.

The value-only path drops keys and headers.

Distributed architecture

Separate the runtimes.

Share Java rules, not active beans.

Choose a connector or a remote contract.

Choose the deployment model before choosing the integration mode.

The number of partitions sets the limits of horizontal elasticity

Adding pods only adds parallelism if there are still partitions to assign to them.

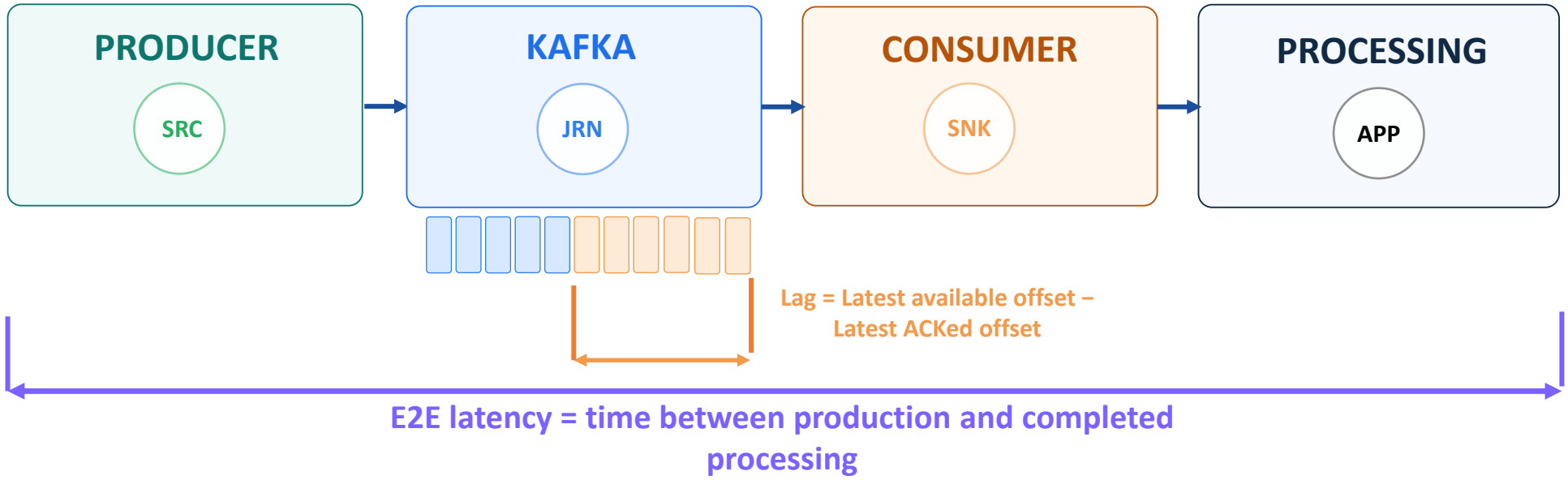
Useful parallelism = min(partitions, pods × threads per pod)

Example: 8 partitions	Requested threads	Maximum active threads
1 pod × 2 threads	2	2
4 pods × 2 threads	8	8
5 pods × 2 threads	10	8 (2 inactive threads)

Excess threads create neither partitions nor useful capacity.

Latency and Lag … The EDA pain point

Two complementary measures: lag indicates backlog volume; latency indicates the real time impact.



Key takeaway: Lag = backlog in messages/offsets. Latency = delay in processing time. Correlated, but not equivalent.

Changing consumers changes partition ownership

Elasticity, restart, and failure are part of a group's normal operation.

Triggers

A pod arrives.
A pod stops.
A consumer exceeds a timeout.

Effects

Partition reassignment.

Possible interruption and recovery from the committed position.

Protections

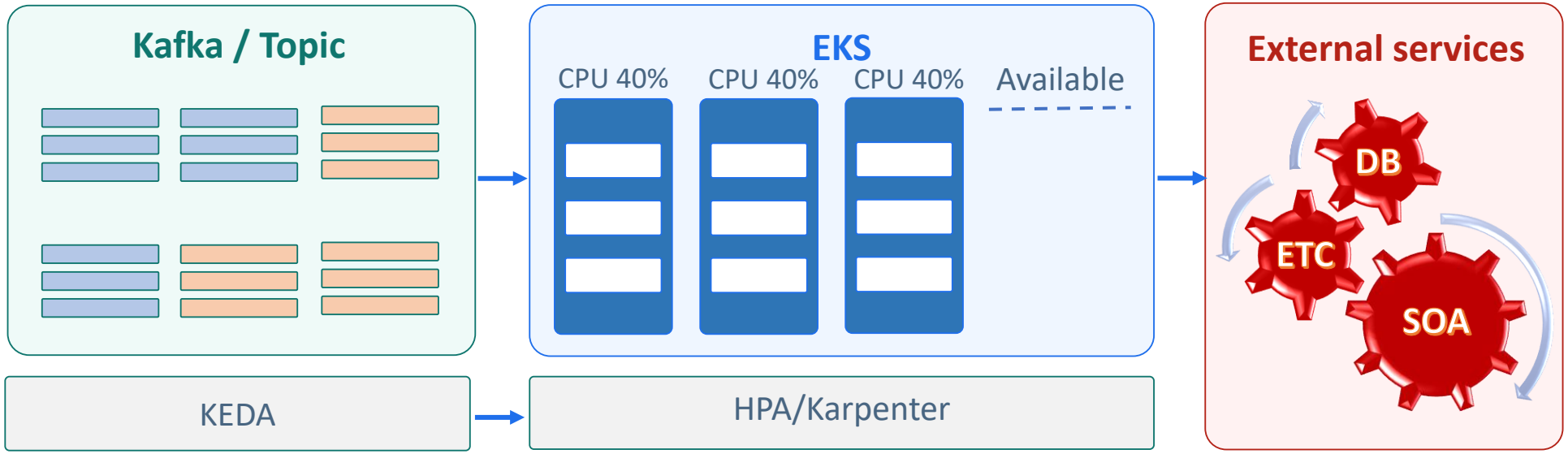
Idempotent processing.
Appropriate batches and timeouts.
Stabilized elasticity.
Appropriate group strategy.

Partition reassignment
Reassignment will route reprocessing (duplicates).

An elastic system must know how to change owners without corrupting state.

EKS/EDA horizontal elasticity

Horizontal elasticity for EDA consumers depends on knowing the LAG.



Horizontal elasticity

CPU/memory HPA fails: high lag without high CPU, partition ceiling ignored, no scale-to-zero.

HPA/Karpenter manages machine instances; KEDA manages lag and consumption.

Pausing is not stopping the consumer

To temporarily suspend work without leaving the group, the sandbox uses pause() / resume().

A console to control operations is not optional!!!

Action	Consumption	Group membership
pause()	Suspend consumption.	The consumer remains a member.
resume()	Resume consumption.	The same instance continues.
stop()	Stop the container.	Exiting triggers a rebalance (possible reprocessing).
start()	Start the container.	Entering triggers a rebalance (possible reprocessing).

Kafka: lightweight container pause. Flink/Sandbox: stop with savepoint (checkpoint). A rebalance triggers reprocessing!!!

Shutdown is part of system adjustments

The grace period must allow in-flight work to finish and resources to close in the right order.

1 **Remove traffic** → Readiness / drainage

2 **Signal shutdown** → preStop / SIGTERM

3 **Finish in-flight work** → Commit / Ack

4 **Close resources** → Before the grace period ends

```
#
server:
  shutdown: graceful
#
  lifecycle:
    timeout-per-shutdown-phase: 45s
#
```

application.yml

Shutdown in progress.

```java
public class KafkaPipelineListener {
    @KafkaListener(id = KafkaConstants.CST_LISTENER_PROCESSOR,
                   topics = KafkaConstants.CST_TOPIC_KAFKA_REQ,
                   groupId = KafkaConstants.CST_TOPIC_GROUP,
                   containerFactory = "kafkaListenerContainerFactory")
    public void listen(ConsumerRecord<String, String> record, Acknowledgment ack,
                       @Header(KafkaHeaders.RECEIVED_TOPIC)     String topic,
                       @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                       @Header(KafkaHeaders.OFFSET)             long offset) throws Exception {

        // If the application is shutting down, stop processing immediately. The record is
        // neither processed nor acknowledged, so its offset is not committed and it will be
        // redelivered after restart (at-least-once).
        if (!smartLifecycleSrv.isRunning()) {
            logger.warn("Application is shutting down – skipping record - topic={}, partition={}, offse
                topic, partition, offset);
            return;
        }

        // Per-record processing (per-client lock + transactional processor + completion count)
        // is shared with the Flink sink path; see PipelineSrv.handle(). Any exception propagates to
        // the container's DefaultErrorHandler, which retries with a bounded back-off and ultimately
        // routes the record to the DLT. The JPA transaction in the processor rolls back on failure,
        // so no partial DB state is committed.
        pipeline.handle(record.value());

        // Commit the offset only after a successful, committed processing.
        // (at-least-once: a crash before this point replays the message.)
        ack.acknowledge();
    }
```

For Kafka producers and consumers, graceful shutdown is a critical requirement, not an optional optimization.

A clean shutdown reduces reprocessing; idempotence makes reprocessing possible.

Three probes answer three different questions

A service access failure and an unrecoverable process do not call for the same reaction.

Startup

Has startup completed?

Do not judge a JVM too early during initialization.

Liveness

Is the process healthy?

A failure can cause it to restart.

Readiness

Can it receive traffic?

A failure removes the pod from the service endpoints.

```yaml
management:
  server:
    port: 8081
  endpoints:
    web:
      exposure:
        include: "info,health,prometheus"
  prometheus:
    metrics:
      export:
        enabled: true
```

File: application.yml.

For Kafka producers and consumers.
Internal state management is a critical requirement.

Health: a quick decision. Metrics: a measure of behavior over time.

A contract turns a topic into an understandable interface

Governance makes the meaning of the data, responsibilities, and usage conditions explicit.

Business meaning

Definition and event examples.

Ownership

Responsible team and support.

Schema

Version and compatibility rules.

Dependencies

Producers, consumers, and lineage.

Service

Freshness, availability, and quality.

Protection

Access, classification, and retention.

A topic without a contract or owner creates a dependency that is difficult to operate.

Ownership and platform are complementary

The guide does not present Data Mesh and Data Fabric as two competing products.

Data Mesh: who and why?

Domain ownership.
Data as a product.
Self-service platform.
Federated governance.

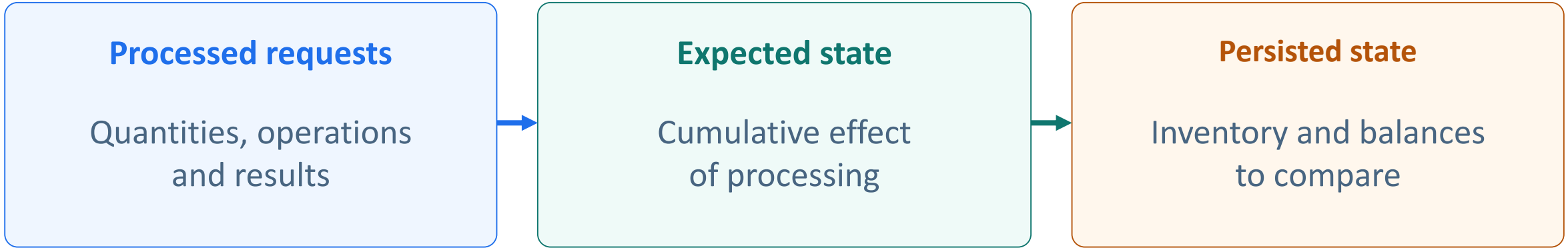
Data Fabric: how to connect?

Integration and connectors.
Catalog and lineage.
Policies and quality.
Shared access and observability.

Kafka can support these approaches; it replaces neither ownership nor governance.

A consumed message does not prove that the state is correct

Corroboration compares representations of the same facts and looks for discrepancies.



Concrete control: do executed requests reconcile with inventory movements and the customer balance?

Idempotence: do not apply twice. Corroboration: verify the overall result (integrity).

Data corroboration in EDA

Event delivery never guarantees that systems remain consistent with each other.

01 Frequent

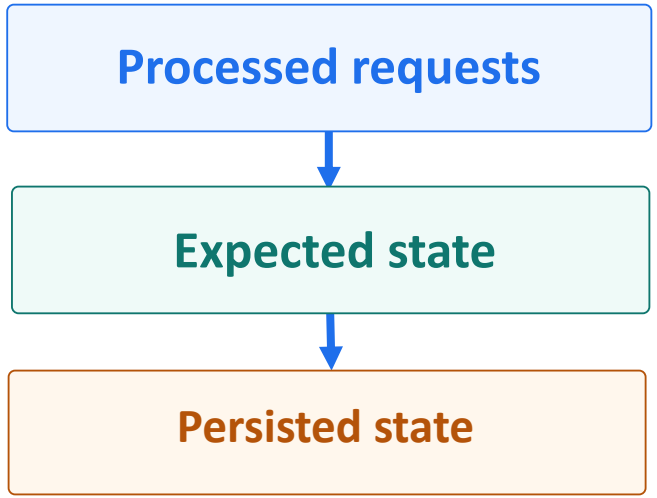
Result counts.
Checksums.

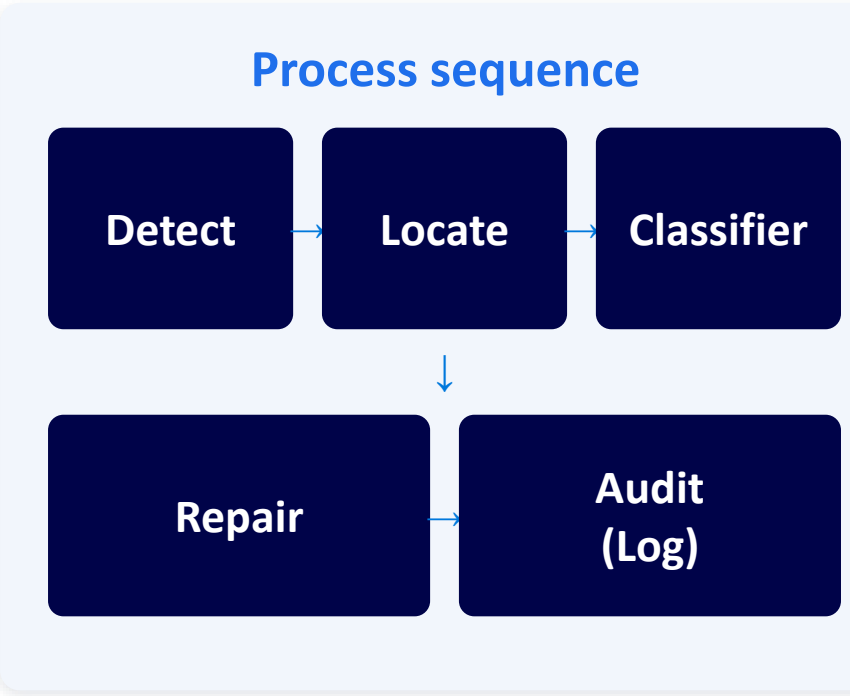
02 Periodic

State snapshots.
Business invariants.
Expected/actual comparison.

03 Deep-dive

Reconciliation replay.
Ghost consumer.
Gap audit.





Repair without losing the audit trail; replay without reapplying effects twice.

They are not three rungs on a maturity ladder

The documentation compares complementary integration styles that place complexity in different locations.

Dimension	ETL (batch)	SOA (synchronous)	EDA / Kafka
Interaction	Scheduled tasks.	Request-response.	Publication and consumption.
Sought value	Bulk movements and preparation.	Immediate result for the requester.	Independent consumers, buffer, and replay.
Complexity to master	Schedules, lineage and quality.	Availability, calls and compensation.	Ordering, duplicates, DLT and eventual consistency.

The right question: which mechanism meets the need with manageable complexity?

Justify EDA by its value, not by the platform choice

Compare options over the same horizon and make assumptions, costs, and benefits explicit.

TCO: the full cost

Platform and environments.
Engineering and operations.
Accuracy, testing, and reconciliation.
Governance and security.

ROI: measurable benefits

Faster reaction.
Event reuse.
Independent evolution.
Replay, audit, and restoration.

Architecture: needs + options + guarantees + TCO/ROI + owners + review criteria.

The sandbox makes limits visible; it does not erase them

Ease of local execution and production architecture guarantees are two distinct objectives.

For local learning	Toward a production target
Embedded engines and database.	External services, shared state, and explicit durability.
In-JVM Flink bridge and temporary storage.	Target runtime, connectors, and validated restore.
Educational options and dashboards.	Security, contracts, monitoring, and operable procedures.
Intentional concurrency anomalies.	Protected invariants and tested failure scenarios.

Every guarantee must be tied to a scope, an assumption, and a proof.

Four experiments to connect concepts to observations

Demo

Prepare a comparable starting state, change only one variable, and explain what is observed.

01 With / without key / without lock

Does ordering by product change inventory?

02 Replay and idempotence

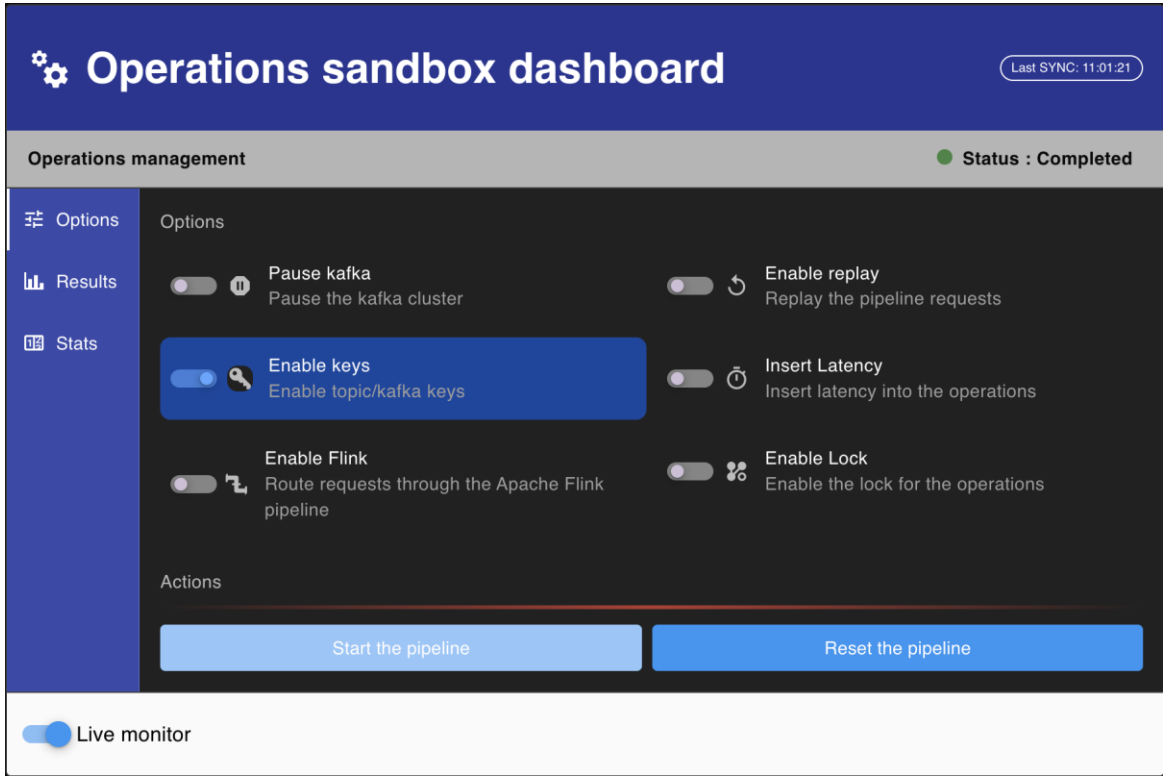
Which effect must not be applied twice?

03 Kafka pause vs Flink pause

What does each recovery mechanism preserve?

04 Business verification

Do the requests explain the final inventory?



Observe → formulate a hypothesis → modify one variable → explain the result (perform manual corroboration).

A resilient EDA is built end to end

The value of the project is to make visible the responsibilities that transport alone does not handle.

Delivering a message, applying an effect, and proving the result are three distinct responsibilities.

01	Order by entity	02	Define atomicity boundaries	03	Tolerate duplicates
04	Prepare recovery	05	Verify the data	06	Justify complexity

Discussion: what guarantee are you claiming, and how would you demonstrate it?

Find the journey chapters in the project

Paths are relative to doc/. Each slide also has detailed sources in the notes.

Project presentation

README.md

Data and journal

database.md

Delivery and reliability

reliability_and_delivery_semantics.md

Persistence and transactions

persistence_and_transaction_patterns.md

Architecture and topics

architecture_and_topic_design.md

Report comparison

reports.md

Acknowledgement and idempotence

consumer_acknowledgement_and_idempotency.md

Embedded Kafka, Flink, and Hazelcast

sandbox_guides.md

References for this deck: project documentation.

Continue from technical resilience to trust

The progression links execution mechanisms, operations, governance, and architecture choice.

Sandbox Flink path

flink_guides.md

Lifecycle and operations

lifecycle_and_operations.md

EKS elasticity

horizontal_elasticity_keda_karpenter.md

Corroboration

data_corroboration.md

ETL / SOA / EDA choices and TCO/ROI

conclusion_etl_soa_eda_tco_roi.md

Flink / Spring Boot integration

flink_springboot.md

Distribution and shutdown

dist_scale_and_shutdown.md

Governance and observability

governance_and_observability.md

Data Mesh / Data Fabric

datamesh_datafabric.md

Inbox and Outbox Design Patterns

Inbox_outbox_patterns.md

References for this deck: project documentation.

Read JRN_EVENT without confusing its responsibilities

The journal distinguishes a processing identity, correlation, nature, and outcome.

Field	Question it answers
JRN_EVENT__ID	Primary key. Is this the surrogate key?
CONSUMER_ID + EVENT_ID	Which consumer processed which event?
BATCH_ID	Which publication does it belong to?
EVENT_TYPE	First delivery or deliberate replay?
RESULT	PENDING, BACK_ORDER, EXECUTED, or ERROR?
PAYLOAD	What content and processing chronology?

Local dependencies to replace in production

The sandbox project is an educational tool.

Composant	Sandbox fourni	Cible distribuée
Base de données	HSQLDB fichier local	PostgreSQL / RDS partagé
Kafka	Broker embarqué, RF=1	Cluster externe, réplication ≥ 3
Hazelcast	TCP/IP localhost	Découverte Kubernetes + cohérence choisie
Flink	MiniCluster + pont JVM	Cluster externe + sink durable
Pilotage	Événements Spring locaux	Commandes distribuées et runId

Sandbox limitations

The sandbox project is an educational tool. Some choices were made to facilitate learning and/or implementation.

Limitation	Description
Pause	The pause feature assumes a single process. The implementation must be refined for PROD.
Replay	Replay is very simplistic and intended for demonstration, not a real implementation. To be refined for PROD.
RDBMS Model	The model is intentionally simplistic. Customer: customer, account, balance, trx. Inventory: category, model, etc.
Sandbox	Only the Kafka Sandbox is production quality. Flink is a base, and Hazelcast requires refactoring: Datagrid Sandbox.
Flink / In-JVM	Exploratory in-JVM implementation to avoid having to redo the implementation in Spring Boot.
Token / EVT	To facilitate implementation of the “replay”. The token does not contain the “version” field.
Security	The security is intentionally simplistic, with hard-coded values. The JWT token is stored in the browser.